

简书

首页

下载APP

搜索

Q

Aa

beta

登录

注册

《深入理解计算机系统》| 存储器层次结构

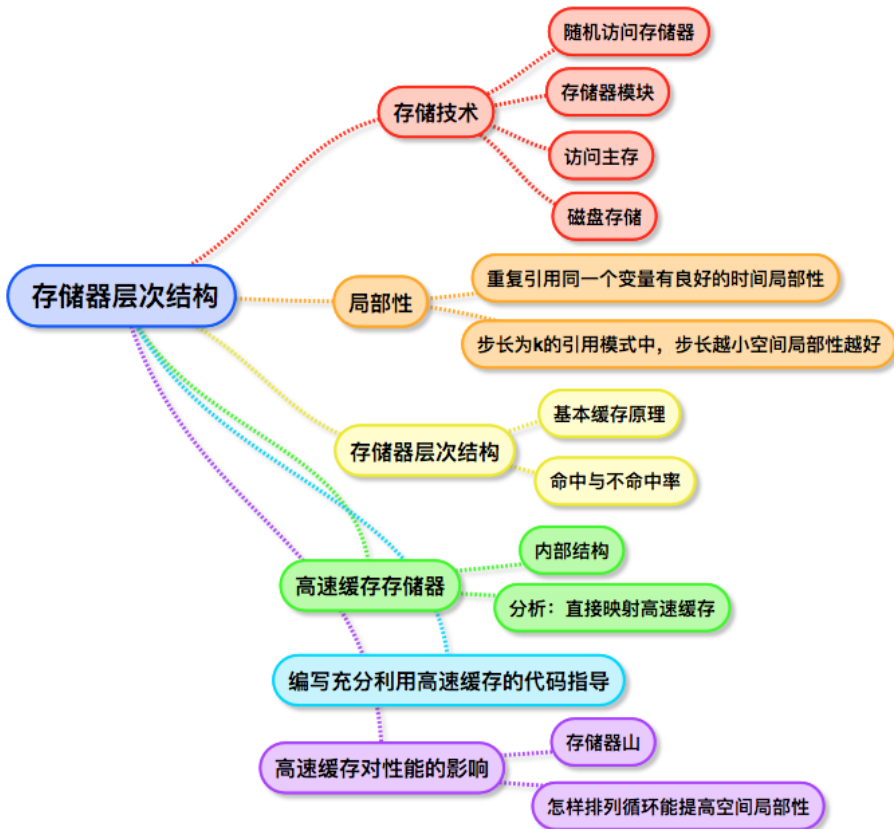


唐鱼的学习探索

关注

0.235

2018.03.18 05:12:19 字数 5,231 阅读 1,847



目录

在本章中，我们会先了解存储技术（SRAM\DRAM\ROM\旋转固态硬盘），描述这些存储器是如何被组织成层次结构的。接下来会谈什么是拥有良好局部性的程序以及编写这样的程序需要注意的问题。然后我们开始探究本质，为什么说拥有良好局部性的程序会执行的更快。就要求我们要学习高速缓存，并教会大家理解程序的局部性的真正意义，使得你自己不仅仅遵守规则，而是了解其内部原理获取更大的自由。

1.1 存储技术

① 随机访问存储器

静态RAM：（SRAM）用作高速缓存，通常只有几兆，在CPU芯片上、下；硬件设计中，将每个位存在一个双稳定的存储单元中，如下图所示，只有在两边的时候保持稳定性：



唐鱼的学习探索

关注

总资产 29 (约2.80元)

如何高效的准备一次考试

阅读 1,735

都9102年了，你还不知道anki是什么

阅读 102

推荐阅读

面试问我，创建多少个线程合适？
我该怎么说

阅读 15,572

三面字节跳动被虐得“体无完肤”，
15天读完这份pdf，终拿下美团研发

阅读 77,766

一款SQL自动检查神器，再也不用担心SQL出错了，自动补全、回滚等功

阅读 13,529

MySQL索引原理

阅读 5,154

一套可复用的方法论！从0-1搭建数据团队，看这篇就够了

阅读 4,682



写下你的评论...

评论1

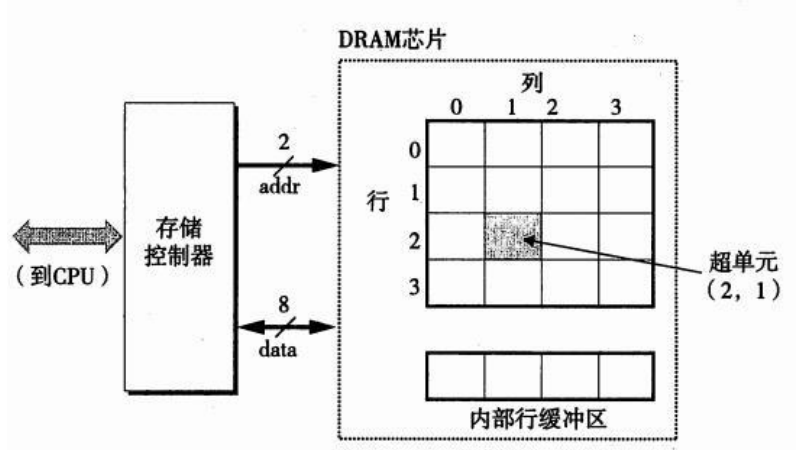
赞11

...



动态RAM：（DRAM）用作主存（我们通常说的机器的内存），通常几百、几千兆。每个单位使用一个电容和一个访问晶体管构成，容易被干扰，有的加入有纠错码。系统需要周期性读出，然后刷新重写存储器的每一位。

DRAM详细构造图：

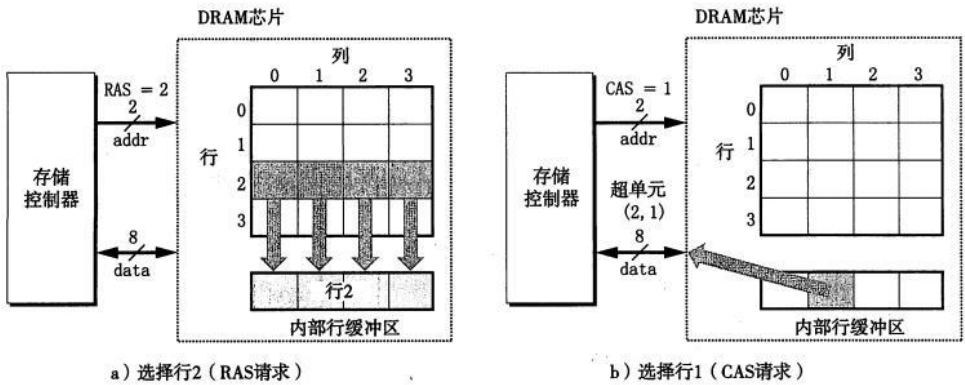


一个128位的16X8的DRAM构造视图

DRAM芯片被分割成16个超单元，每个超单元又由w个DRAM单元组成，一个16*w的DRAM共存储16w位信息。超单元被组织成一个4*4的阵列，图中的超单元地址用（2，1）表示。信息通过引脚流入和流出芯片，每个引脚携带1位信号，图中有两组引脚，其中data引脚为8个，能传出或接受来自芯片的一个字节数据，还有两个addr引脚，携带2位的行列超单元地址。

访问示例（我们来看看是如何访问超单元（2，1）处的内容）

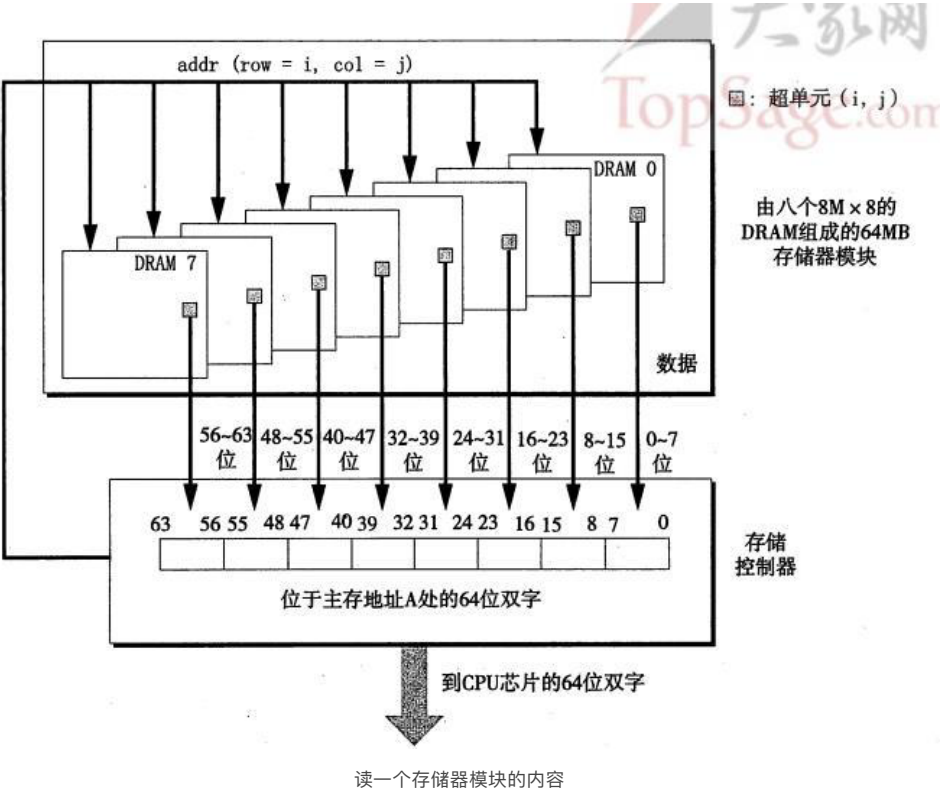
分析：我们首先来思考一个问题，为什么要组成一个二维的数组，而不是一位数组，这样访问的速度就快很多啊。以上图为例，我们如果要建造一个128位的DRAM，我们用一维数组实现的话，我们需要提供大量的地址引脚来提供访问（硬件设计上太费了）。



读取一个超单元

为了加快二维数组的访问，存储控制器在读取（2，1）处的内容的时候，使用addr先发送行地址2到DRAM芯片中，拷贝整个第二行的内容到内部缓冲区中，然后发送列地址1，从内部行缓冲区中读取1的地址内容通过data发送到存储控制器中去。

② 存储器模块

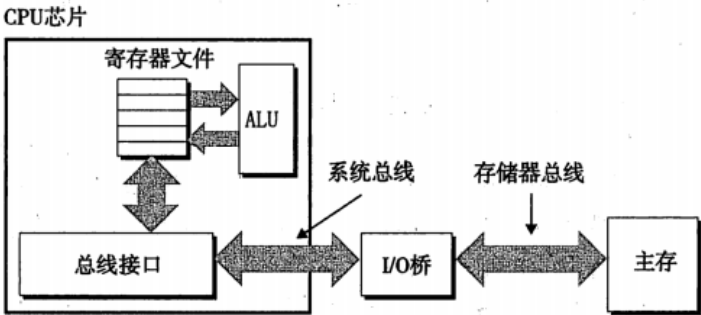


如图所示是一个64M的主存，芯片编号0-7，每个芯片存储8M的数据，存储器模块将其组合起来，聚合内存。将每个芯片的超单元映射成主存地址A的各个字段。这样控制器收到一个主存地址A的时候，存储控制器将其选择包含的具体芯片，将A转换成（i，j）的形式，然后将（i，j）发送到芯片模块中开始取数据。

备注：存储在ROM设备中的程序通常称为固件，当一个计算机系统通电以后，它会运行存储在ROM中的固件。

③ 访问主存（读事务、写事务）

数据流通过总线，在CPU和DRAM中传递数据，总线能携带：地址、数据和控制信号。



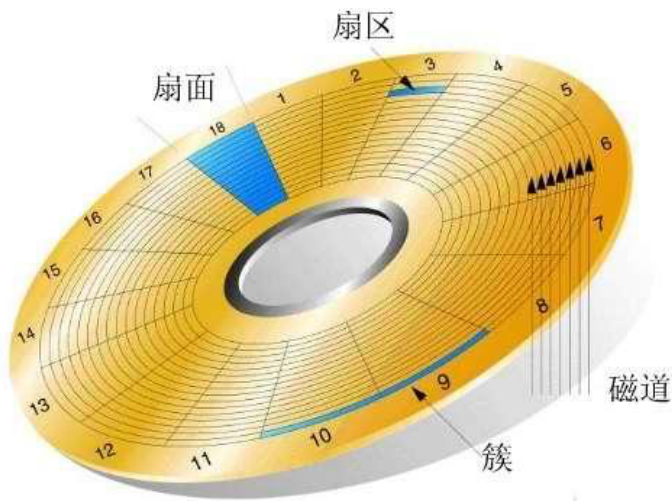
CPU和主存的总线构照图

读事务：考虑当我们执行，`movl A, %eax`的情况，地址A的内容会被加载到eax中去，总线发起读事务（分三步）：①CPU将A的地址总线放到系统总线上，桥作为中转点，将地址信号传送到存储器总线上；②主存感觉到了存储器总线上的地址信号，从存储器总线上读地址，并从主存中取出相应的数据，写入到存储器总线上，桥将数据专递到系统中线中去；③CPU感觉到了系统总线上的数据，将数据拷贝到eax中。

I/O桥作为中转，将地址信号从系统总线转到存储器总线，然后又将数据从存储器总线转到系统总线。在这个过程中，CPU始终是从系统总线上发送地址，读取数据，主存始终是从存储器总线上接受地址并发送数据。（写事务是一个逆向过程不做讲解）

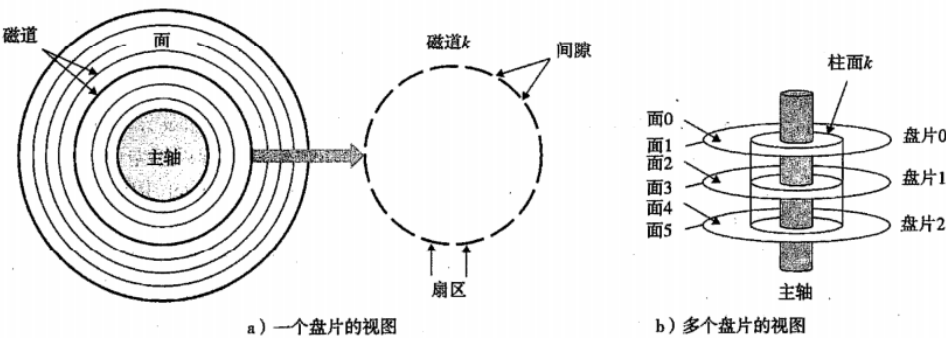
④ 磁盘存储（硬盘）

构造：



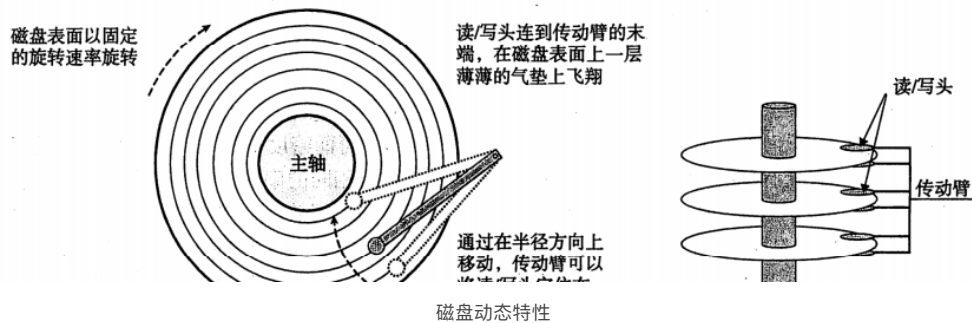
磁盘上的磁道、扇区和簇

磁盘由盘片构成，表面覆盖的有磁性材料，中间是一个主轴，通过旋转读取和记录数据。每组同心圆磁道分割的区域就是一个扇区。扇区之间是有间隙的，如图：



磁盘构造

磁盘读写操作：



磁盘以扇区为单位来读写数据，对扇区的访问时间由三个部分组成：寻道时间、旋转时间、传送时间。以图a为例，当我们要访问同心圆磁道5的内容时，寻道时间是指传动手臂将读写头移动到同心圆第五磁道的时间，旋转时间指的是同心圆5开始读取内容的位置，如果手臂移动到第五磁道的时候读写位置刚过，就要等磁盘旋转一圈之后再读取；传送时间，扇区第一个位处于读写头的时候，读写该扇区的时间。（寻道时间和旋转延迟大致相当）

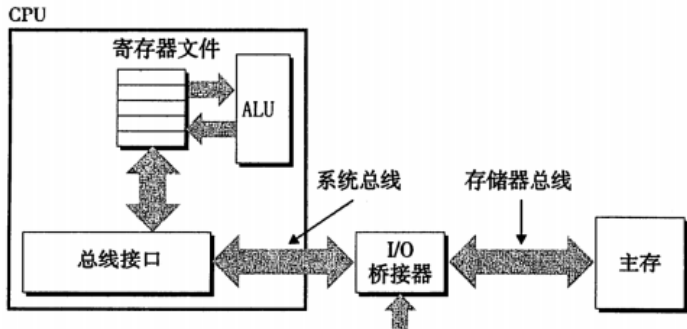
磁盘为什么都是密封的？在传动臂末端的读/写头在磁盘表面高度大约0.1微米处的一层薄薄的气垫上飞翔(就是字画上这个意思)，速度大约为80km/h。这可以比喻成将Sears Tower(译者注，一座位于芝加哥的108层和442米高的摩天大楼)放倒，然后让它在距离地面2.5 cm(1英寸)的高度上飞行环绕地球，绕地球一天只需要8秒钟!在这样小的间隙里，盘面上--粒微小的灰尘都像一块巨石。如果读/写头碰到了这样的一块巨石，读/写头会停下来，撞到盘面——所谓的读/写头冲撞(head crash)。

逻辑磁盘块：

以我们正在使用的计算机为例，当我们安装的有ghost软件开始备份的时候，就会要求选择备份文件的位置，我们看到的是1.1-1.6左右的可以选择的硬盘，实际上磁盘虽然进行了分区，但本质上仍然只有一块磁盘。在磁盘中有一个小固件，磁盘控制器，维护着磁盘扇区之间的映射关系。假设我们要打开E：上的一个文件，控制器就会执行一个快速表查找，将该处的内容翻译成（盘面、磁道、扇区），等到传动臂移动到正确的位置时，将内容读到一个缓冲区，然后拷贝的主存中去。

逻辑块的作用：当我们对磁盘进行分区以前都要求我们进行格式化，这样做是让磁盘控制器，读取磁盘的基础内容，同时建立备用扇区，当一个扇区不能访问的时候，磁盘控制器启用备用扇区，这样使得磁盘更健壮，不会因为一点点损坏就不能使用了。备用扇区可能相当的大。

连接I/O设备



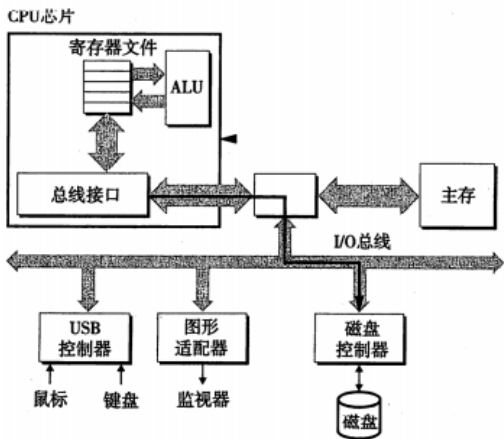
总线结构图：CPU、主存、I/O设备

I/O总线也是通过桥和CPU相连，这样的设计有更大的兼容性，比如USB（通行串行总线）可以连接多个不同设备（打印机、鼠标、键盘），传送600M/s的数据（usb3.0）。图形卡（GPU）代替CPU在显示器上像素显示。特别的来讲，磁盘是通过主机总线适配器同io总线相连的。

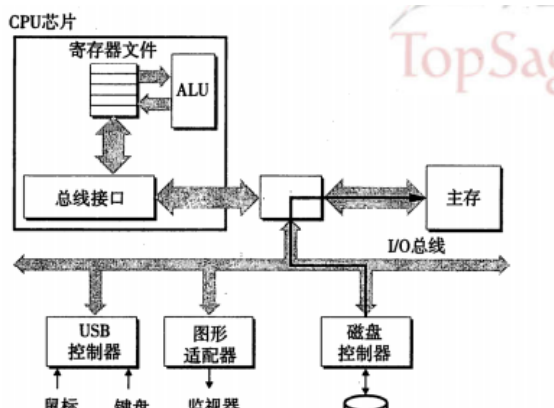
访问磁盘：（磁盘-主存-CPU）

对磁盘的数据访问，并不是直接从磁盘到CPU，而是通过主存作为桥梁，达到快速访问。我们现实生活中的桥，貌似也是这个作用。

当我们要读取磁盘0xa0的内容，cup发出三道指令：1] 发送一个命令，要求读磁盘内容，要求读完以后报告给CPU（中断）；2] 指明要读取的具体逻辑块号码；3] 指明拷贝到主存的地址。



a) CPU通过将命令、逻辑块号和目的存储器地址写到与磁盘相关联的存储器映射地址，发起一个磁盘读



为什么要使用中断：一个1GHz的CPU时钟周期是1ns，读磁盘的16ms的时间内，可以执行1600万条指令，这个时间如果只是等待的话就太浪费了。CPU发起读指令以后，就不用管了，等到磁盘控制器将内容全部COPY到主存中，磁盘控制器发起一个中断，告诉CPU，不要做自己的其他事情了，你之前让我读磁盘的内容已经全部读到主存中去了。

总结：现代计算机频繁的使用SRAM的高速缓存，试图弥补越拉越大的存储器与CPU之间的差距，我们接下来就来看看局部性这一属性是如何能弥补速度差的。

1.2 局部性：以前用过的我还接着用

我们讲存储器体系结构就会很好的理解局部性，简单的来说，我们的主存就是我们为了提高我们磁盘文件的一个高速缓存，因为我们知道这一时刻访问到磁盘的数据可以下一时刻也会被访问，这一位置被访问的数据，邻居位置也可能被访问。这也就是我们通常说的：时间局部性和空间局部性。

对程序数据访问的局部性

M N

$a_{[3][5]}$

0,0	0,1	0,2	0,3	0,4
1,0	1,1	1,2	1,3	1,4
2,0	2,1	2,2	2,3	2,4

双重嵌套循环

按行优先顺序存储。

访问按行进行。

假设我们有这样的一个二维数组：

```
1 int sumarrayrows(int a[M][N])
2 {
3     int i, j, sum = 0;
4
5     for (i = 0; i < M; i++)
6         for (j = 0; j < N; j++)
7             sum += a[i][j];
8     return sum;
9 }
```

Address	0	4	8	12	16	20
Contents	a_{00}	a_{01}	a_{02}	a_{10}	a_{11}	a_{12}
Access order	1	2	3	4	5	6

我们遍历每个数组求和，这样的sum变量有很好是时间局部性，因为我们访问过一次，又接下来继续在访问。由于二维数组是按照行的顺序存储的，按照步长为1的求和，也使得程序有很好的空间局部性。

如果我们作一些改变，使得程序按照列的顺序访问：

```
int sumarraycols(int a[M][N])
{
    int i, j, sum = 0;

    for (j = 0; j < N; j++)
        for (i = 0; i < M; i++)
            sum += a[i][j];
    return sum;
}
```

Address	0	4	8	12	16	20
Contents	a_{00}	a_{01}	a_{02}	a_{10}	a_{11}	a_{12}
Access order	1	3	5	2	4	6

❖交换j和i的循环位置，使得程序按照列的方式求和，那么程序的局部性就相当的差了。

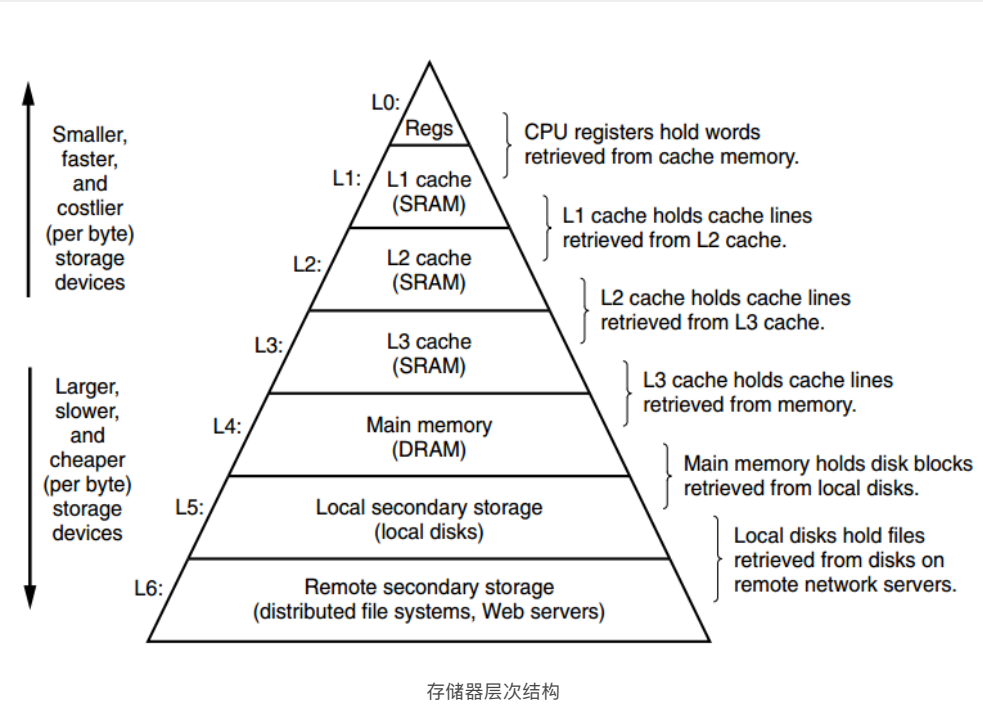
总结：

1.重复引用同一个变量的程序有良好的时间局部性；

2.具有步调长度为k的引用模式程序，步调越小，空间局部性越好。

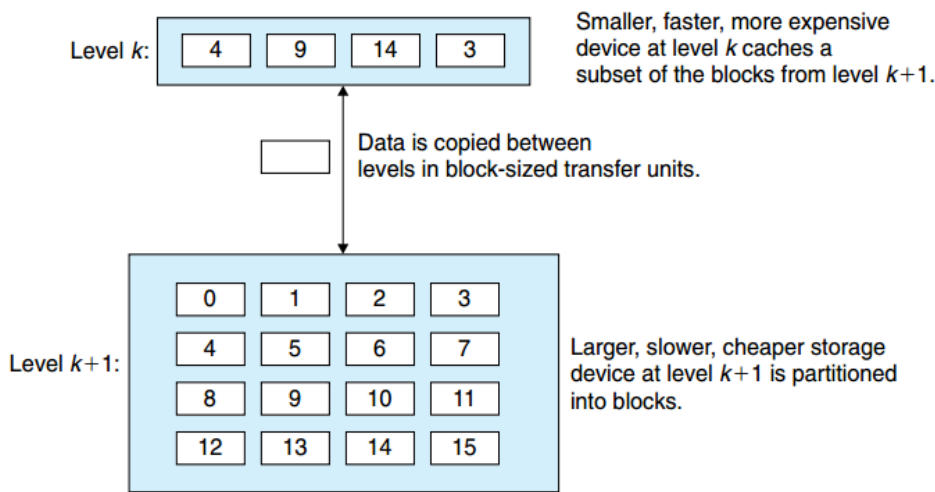
我们一直在说具有良好的局部性的程序将获得更快的运行速度，究竟是什么原因导致了这种运行速度的提升，我们将学习高速缓存中的命中率和不命中率来量化局部性的概念，这就是我们接下来要讲到的内容了。

1.3 存储器层次结构：理解命中率和不命中率



越往上，代表的是访问速度越快，当然存储容量小，价格也非常的高。越往下，意味着访问速度越慢，存储容量大，价格相对便宜。通常我们CPU的寄存器是L1的高速缓存，L1是L2的高速缓存，以此类推。

基本缓存原理：我们来看一个片段，下图为L3作为主存的高速缓存：



基本缓存原理

上图我们把 $k+1$ 理解为主存，被划分为16个块来存储数据，块的大小是固定的。我们把 K 层理解成L3高速缓存，任何时刻L3就是主存的一个子集。上图我们能看出，L3只能保存4个块的数据，块的大小保持和主存的大小一样的。上图中我们看到，L3中保存的是主存中的4，9，14，3的数据。那么什么又是命中率和不命中率呢？

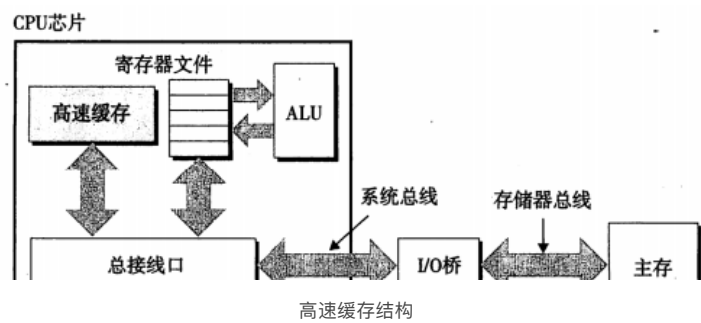
缓存命中：当程序需要第 $k+1$ 层数据块14的时候，程序会在当前存储的 k 层，寻找块14的数据，刚好14在 k 层的话，就是一个缓存命中，这比从 $k+1$ 层读取的速度要快很多。

缓存不命中：当程序需要访问到块12的时候，在 k 层没有该数据块，就是一个缓存不命中，这时候就会从 $k+1$ 层中读取块12将其替换到 k 层的一个数据块（覆盖或驱逐一个已有的数据块）。程序还是从 k 层访问块12。

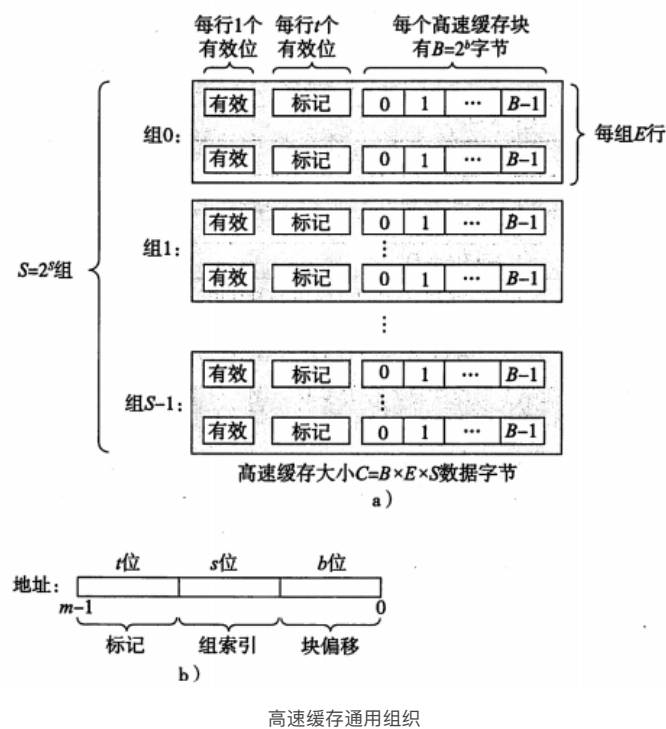
放置策略：如果我们从 $k+1$ 层中获得的数据随机的放置在 k 层，这样的随机放置就会导致访问的效率降低，我们的放置策略是块 i 必须放置在 $(i \bmod 4)$ 中，也就是0，4，8，12会映射到同一个 k 层的块0中。这就会导致一个冲突不命中，也就是说如果程序交替请求 $k+1$ 层的0，4块，由于会一直映射到 k 层的0块中，这时候虽然 k 层有空余的缓存，但还是每次不命中。

总结：利用时间的局部性，同一数据对象可能会被多次使用，一旦一个数据对象在第一次不命中的时候被拷贝到缓存中，我们会期望在接下来的访问中有一系列的命中率。利用空间的局部性，由于一个数据块并不仅仅只有一个数据，而是一系列数据块的集合，我们访问到块子集 a 的时候，可能会继续访问块的子集 b 。

1.4 高速缓存存储器（集成在CPU内部的一个部件L1、L2、L3三级缓存）



① 通用高速缓存存储器内部结构



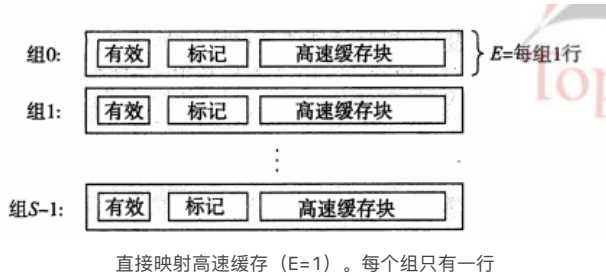
高速缓存是一个数组，每个组包含一个或多个行，每个行有一个有效位、一个标记位，以及数据块。我们进行访问的地址结构就是：t的标记位+s个组索引+b个块偏移；

基本参数	
参数	描述
$S=2^s$	组数
E	每个组的行数
$B=2^b$	块大小（字节）
$m=\log_2(M)$	（主存）物理地址位数

衍生出来的量	
参数	描述
$M=2^m$	存储器地址的最大数量
$s=\log_2(S)$	组索引位数数量
$b=\log_2(B)$	块偏移位数数量
$t=m-(s+b)$	标记位数数量
$C=B \times E \times S$	不包括像有效位和标记位这样开销的高速缓存大小（字节）

基本参数术语一览表

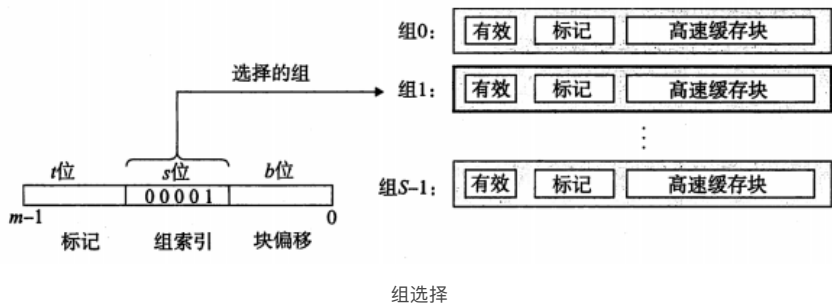
② 直接映射高速缓存（每个组只有一行的简单访问模式）



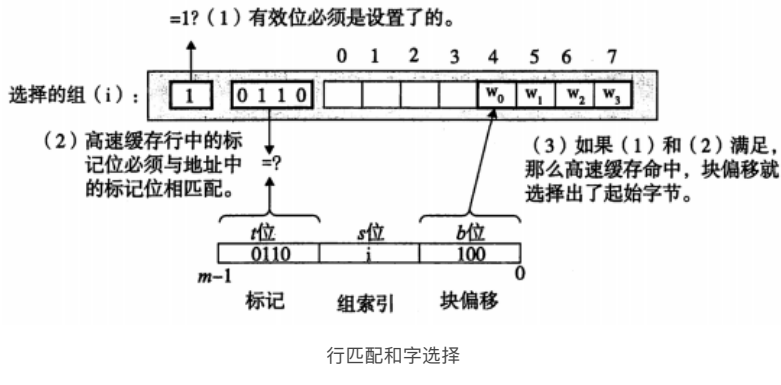
高速缓存确定一个请求是否命中，然后抽出请求字的过程分三步：

(举例：直接映射高速缓存的抽取请求字的过程就像我们投递快件一样，组索引其实就像我们的邮政编码，比如我们这里的510824，然后找到编码的组，也就是我的大位置（xx县），然后看标记上写的具体xx小区x栋楼，并且核实该地址是否有效（有效位1），两项都满足条件以后将该快件给快递员投递，快递员到达具体xx小区x楼的时候就根据门牌号（偏移位）敲开你家的门。binggo，快递到达）

1> 组选择：很好理解，就是地址位中的组索引匹配高速缓存中的组

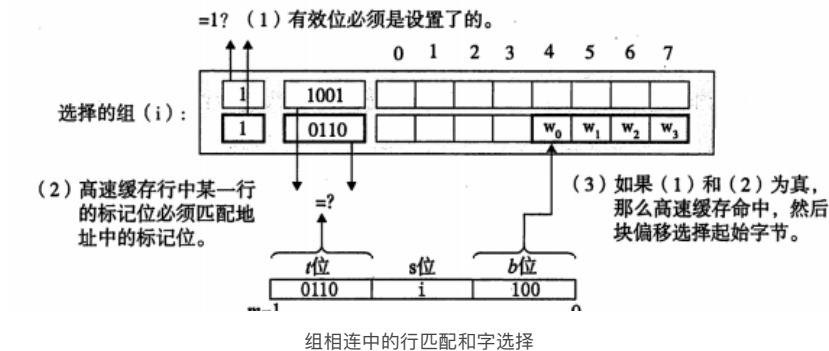


2> 行匹配和字抽取

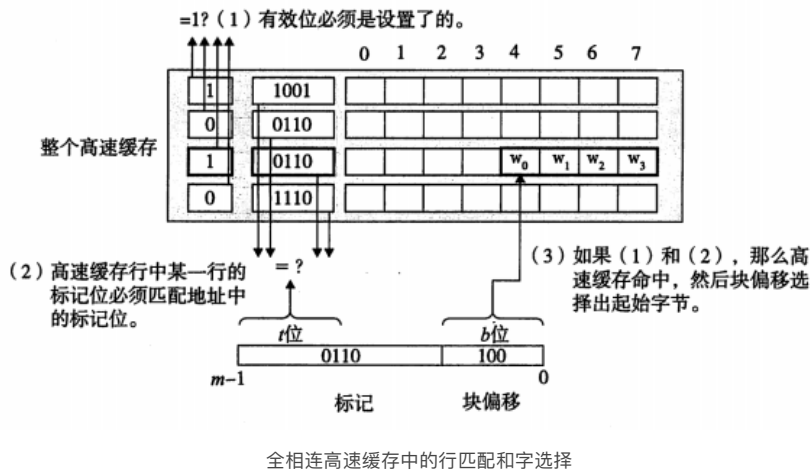


行匹配主要是对有效位进行匹配，和标记位与高速缓存中的标记位一致，这就是一个命中。最后的字抽取就简单的，只是看地址后面的偏移值。

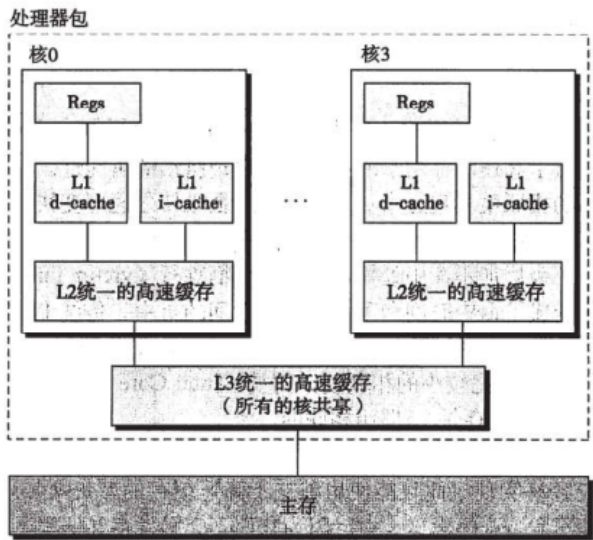
② 组相连高速缓存（每组多于1行的高速缓存）大致方法仍然一样



③ 全相连高速缓存（只有一个组）：只适合小规模的高速缓存（翻译备用缓冲器）



④ 结构剖析（真正意义上的高速缓存）



Intel Core i7高速缓存层次结构

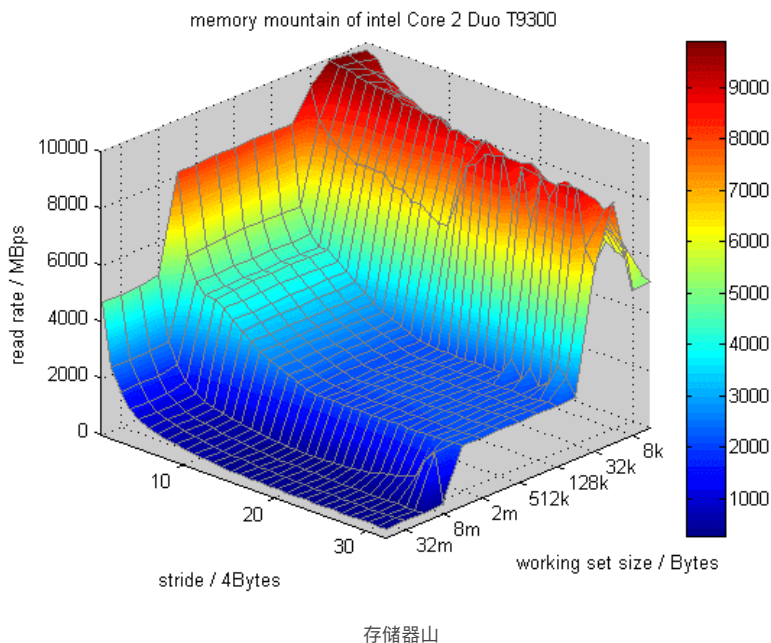
在实际的商用CPU中，将高速缓存分为d-cache数据高速缓存，i-cache指令高速缓存和同一的高速缓存，i7的架构中我们可以看出，L1分为数据和指令高速缓存，共享L2高速缓存，同时每个核共享L3高速缓存

1.5 编写高速缓存友好的代码指导意见

- 1.对局部变量的反复引用是好的，因为编译器能将它们缓存在寄存器文件中；
- 2.步长为1的引用模式是最好的；
- 3.多维数组的访问，注意使用行优先模式。

1.6 高速缓存对程序性能的影响

① 存储器山



存储器的性能不能简单的用一个数字来描述，如果实在要形容的话，是一座时间局部性和空间局部性构成的山。山峰和低谷的差别不是一个数量级。明智的程序员会试图构造运行在山峰的程序而不是低谷。我们来看看这座存储器山是如何画出来的？

测试核心代码：

```
for (i = 0; i < elems; i += stride) {  
    result += data[i];  
}
```

读取数组的内容到寄存器中

这段代码所做的事情，就是将data数组的内容依次读取到CPU的寄存器中。其中elems代表的是data的工作集大小也就是size时间局部性，代表Y轴；而stride（步长）代表的是横轴X；Z轴表示吞吐量，Mb/s。越往上吞吐量越大（红色部分）。我们反复以不同的size和stride值调用我们的核心测试代码，就会得到如上图的存储器山。

最高处的红色山峰为L1，由于工作集（size）很小，能全部保存在L1高速缓存中，所以这时候

即使stride很长，对于性能也没太大的影响。

L2和L3、主存随着stride的增加有明显的坡度，空间局部性下降。特别明显的是，主存的蓝色山峰，即使工作集很大（时间局部性极好）其stride（空间局部性）的影响也相当的明显，最高与最低处相差7倍。也就是告诫我们，即使时间局部性无法改变了，空间局部性也可以使得程序的性能极大的提高。

② 从新排列循环以提高空间局部性

考虑一对2*2数组的相乘的问题：

$$\begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}$$

$$c_{11} = a_{11}b_{11} + a_{12}b_{21}$$

$$c_{12} = a_{11}b_{12} + a_{12}b_{22}$$

$$c_{21} = a_{21}b_{11} + a_{22}b_{21}$$

$$c_{22} = a_{21}b_{12} + a_{22}b_{22}$$

数组A乘以数组B等于数组C

矩阵的乘法是由三层嵌套的循环构成的，我们假设i是数组A的循环计数，j是数组B的循环计数，k是数组C的循环计数：

(a) Version *ijk*

```
code/mem/matmult/mm.c
1  for (i = 0; i < n; i++)
2      for (j = 0; j < n; j++) {
3          sum = 0.0;
4          for (k = 0; k < n; k++)
5              sum += A[i][k]*B[k][j];
6          C[i][j] += sum;
7      }
```

(c) Version *jki*

```
code/mem/matmult/mm.c
1  for (j = 0; j < n; j++)
2      for (k = 0; k < n; k++) {
3          r = B[k][j];
4          for (i = 0; i < n; i++)
5              C[i][j] += A[i][k]*r;
6      }
```

(e) Version *kij*

```
code/mem/matmult/mm.c
1  for (k = 0; k < n; k++)
2      for (i = 0; i < n; i++) {
3          r = A[i][k];
4          for (j = 0; j < n; j++)
5              C[i][j] += r*B[k][j];
6      }
```

三个不同的循环版本

我们看一下不同版本代码分析：

DATE *i* OM OT OW OT OT OS OS *k* NOTES

$$\begin{bmatrix} C_{00}=19, C_{01}=22 \\ C_{10}=43, C_{11}=59 \end{bmatrix} = \begin{bmatrix} a_{00}=1, a_{01}=2 \\ a_{10}=3, a_{11}=4 \end{bmatrix} * \begin{bmatrix} b_{00}=5, b_{01}=6 \\ b_{10}=7, b_{11}=8 \end{bmatrix}$$

C *A* *B*

ijk 版本:

$$i=0, j=0: \begin{cases} k=0: \text{sum} = A_{[0,0]} \cdot B_{[0,0]} = 5 \\ k=1: C_{[0,0]} = \text{sum} = 5 + A_{[0,1]} \cdot B_{[1,0]} \\ \quad = 5 + 2 \times 7 = 19. \end{cases}$$

jki 版本:

$$j=0, k=0: \begin{cases} i=0 \Rightarrow C_{[0,0]} \neq A_{[0,0]} \cdot B_{[0,0]} = 5 \\ i=1 \Rightarrow C_{[1,0]} \neq A_{[1,0]} \cdot B_{[0,0]} = 3 \times 5 = 15 \end{cases}$$

$$k=1: \begin{cases} i=0 \Rightarrow C_{[0,0]} \neq 5 + A_{[0,1]} \cdot B_{[1,0]} = 19. \\ i=1 \Rightarrow C_{[1,0]} \neq 15 + A_{[1,1]} \cdot B_{[1,0]} = 43 \end{cases}$$

kij 版本:

$$k=0: \begin{cases} i=0: \begin{cases} j=0: C_{[0,0]} \neq A_{[0,0]} \cdot B_{[0,0]} = 5 \\ j=1: C_{[0,1]} \neq A_{[0,0]} \cdot B_{[0,1]} = 6 \end{cases} \\ i=1: \begin{cases} j=0: C_{[1,0]} \neq A_{[1,0]} \cdot B_{[0,0]} = 15 \\ j=1: C_{[1,1]} \neq A_{[1,0]} \cdot B_{[0,1]} = 18 \end{cases} \end{cases}$$

$$k=1: \begin{cases} i=0: \begin{cases} j=0: C_{[0,0]} = 5 + A_{[0,1]} \cdot B_{[1,0]} = 19 \\ j=1: C_{[0,1]} = 6 + A_{[0,1]} \cdot B_{[1,1]} = 22 \end{cases} \\ i=1: \begin{cases} j=0: C_{[1,0]} = 15 + A_{[1,1]} \cdot B_{[1,0]} = 43 \\ j=1: C_{[1,1]} = 18 + A_{[1,1]} \cdot B_{[1,1]} = 50 \end{cases} \end{cases}$$

上传分析图我们简单的说一下，核心的思想就是最后一个版本：kij执行的效率高很多。最主要的一点就是，在最后一个版本中，每次循环是按照行优先的顺序一步步最后求得数组C的值。

我们认为下列要求是不言而喻的，来结束这一章节的讲解：

1. 将你的注意力集中在内循环上，大部分计算的存储器访问都集中在这里；
2. 通过按照数据实际在存储器中存放的顺序，以步长为1来访问，空间局部性最优；
3. 一旦从一个存储器中读了一个数据出来，就尽可能多的利用他（kij版本）。



11人点赞 >



 《深入理解计算机系统》



"小礼物走一走，来简书关注我"

赞赏支持

还没有人赞赏，支持一下



唐鱼的学习探索 如果我像一般人一样读那么多书，我就跟他们一样愚蠢了。
总资产29 (约2.80元) 共写了10.4W字 获得530个赞 共463个粉丝

关注

ExpressVPN™ - Download VPN

Access Content from Anywhere, on Any Device. Fast Servers in 94 Countries

写下你的评论...

全部评论 1 只看作者

按时间倒序 按时间正序



justdream
2楼 2018.03.17 16:53

您好，存储器测试的代码还有吗？

 赞  回复

被以下专题收入，发现更多相似内容

-  操作系统
-  操作系统原理
-  《深入理解计算...
-  Linux操作系统

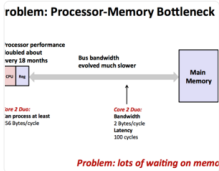
推荐阅读

深入理解存储器层次结构

概述 对于一个简单的计算机系统模型，我们可以将存储器系统看做是一个线性的字节数组，而 CPU 能够在一个常数时间内...

 要上班的斌哥 阅读 1,877 评论 1 赞 9

更多精彩内容 >



《深入理解计算机系统》| 虚拟存储器

虚拟存储器又叫做虚拟内存，我们现在的操作系统普遍都支持了虚拟内存，这样做是因为我们同时运行着太多的程序了，就目前我...

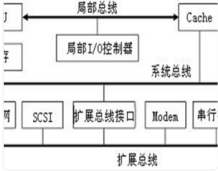
 唐鱼的学习探索 阅读 2,829 评论 1 赞 21



总线

总线 计算机的各个功能部件通过总线连接在一起构成完整的计算机系统，总线是多个系统功能部件之间进行数据传送的公共通路...

 无敌大灰狼me 阅读 1,482 评论 0 赞 9



第6章 存储器层次结构

学习目标 SRAM和DRAM的构成与区别 磁盘存储的结构以及磁盘容量，扇区读取时间的计算 认识局部性原理 认识存储...

 GuangchaoSun 阅读 422 评论 0 赞 0

每一天

清风晨露霞依稀， 阳光灿烂午后晖。 华灯初上霓虹洒， 星夜阑珊戴月归。

 jacky郑 阅读 29 评论 2 赞 1